DP5. Physels: one 32-byte record of solid matter with an exact superquadric shape, drawn without triangle meshes by one exact ray cast per pixel

Defensive publication · version 1 · 23 October 2026 · Author: Kyle Kimberlin (AI assistants helped write it) · Licence: CC BY 4.0

Keywords: physel; primitive-based world representation; fixed-size primitive record; superquadric; superellipsoid; tapered superquadric; bevelled box; frustum; cylinder; cone; implicit surface; ray–primitive intersection; guarded Newton step; non-overshooting root finding; analytic normal; closed-form volume; constructive solid geometry; subtractive primitive; material blend; smallest-three quaternion; logarithmic size code; ray-cast impostor; screen-space bounding rectangle; visibility buffer; deferred shading; conservative depth; depth pyramid; two-round occlusion culling; k-d tree; level of detail; error-bounded stand-ins; signed distance bound; meshless rendering; virtual reality; destructible worlds.

Classification (CPC): G06T 15/06 (ray-tracing); G06T 15/40 (hidden part removal); G06T 17/10 (constructive solid geometry using solid primitives); G06T 15/005 (general purpose rendering architectures); G06T 15/80 (shading); G06T 17/005 (tree description, e.g. octree, quadtree); G06T 9/001 (model-based coding); G06T 2210/12 (bounding box); G06T 2210/21 (collision detection, intersection); G06T 2210/36 (level of detail); G06T 2210/56 (particle system, point based geometry or rendering); G06T 2210/61 (scene description).

Related disclosures in this series (each published separately): DP1, *Sun shadows decided per receiver from the light's side*; DP2, *Certified caches on time laws, and drivers that move frames*; DP3, *Exact integer time, wrap-exact whole-number phases and bit-identical CPU and GPU fixed-point twins*; DP4, *Sound made from the matter*.

1. Field

Computer graphics and physical simulation: storing solid matter as small primitives with an exact shape in a fixed 32-byte record, and drawing them in real time, without triangle meshes, by one exact ray cast per pixel.

2. Problem and known methods

Engines usually describe an object several times: a mesh to draw, a hull for collisions, a recorded sound, baked light. When it breaks, each copy must be rebuilt or faked, and the copies drift apart. What is wanted is one record of matter from which looks, physics, heat and sound are derived: small enough for millions on one graphics card, exact enough that each consumer can state its error.

Known method	What it gives	What it lacks here
Triangle meshes with clustered level of detail (Karis et al. 2021)	Detailed static surfaces	No volume, mass or interior
Voxels; sparse voxel octrees (Laine and Karras 2010)	Regular storage, easy to edit	Curves and thin parts cost many cells

Known method	What it gives	What it lacks here
Distance fields, sphere tracing (Hart 1996; Evans 2015; Aaltonen 2018)	Smooth or exact shapes	Slow on grazing rays; large fields costly to change
Point splats, 3D Gaussians (Zwicker et al. 2001; Kerbl et al. 2023)	Captured appearance	No solid surface or interior
Ray-cast quadric impostors (Gumhold 2003; Toledo and Lévy 2004; Sigg et al. 2006); superquadric glyphs (Kindlmann 2004; Hlawitschka et al. 2008)	Exact per-pixel quadrics	Display glyphs, not stored matter
Guaranteed implicit-surface hits (Kalra and Barr 1989; Mitchell 1990; Knoll et al. 2009; Galin et al. 2020)	Robust first hits	General-purpose, not tuned to one family
Visibility buffer (Burns and Hunt 2013); hierarchical depth (Greene et al. 1993); two-phase culling (Hill and Collin 2011; Haar and Aaltonen 2015)	Shading once per pixel; hidden geometry skipped	Built for triangles
Impostors, level of detail (Maciel and Shirley 1995; Luebke et al. 2002); unified particles (Macklin et al. 2014)	Cheap far stand-ins; one physics	Images or meshes; one particle size

Superquadrics (Barr 1981), taper (Barr 1984), their volumes (Jaklič et al. 2000) and differences of solids by ray spans (Roth 1982) are known too. This disclosure combines and extends them: one fixed-size record with an exact shape from one dialled family, two materials, a blend and a subtractive marker; one solver for the family, closed forms where they exist and a Newton step with a proven no-overshoot guard elsewhere; and drawing of millions of records through screen rectangles, exact casts and a visibility buffer, with culling that never skips anything visible and stand-ins only within a stated pixel error.

3. Summary

A **physel** is a 32-byte record: position, rotation and half-sizes in its object's frame; four shape dials (two roundness exponents, a taper, a bevel) that choose one shape from one superquadric equation; two material ids and a blend; a tint, a seed and flags. Every consumer evaluates the same shape: closed forms for boxes, frustums, ellipsoids, cylinders and cones; Newton steps on exact distances for bevelled ones; for the rest a guarded Newton step, the smallest root of $F + F'\delta - q\delta^2$, which provably never passes the surface. To draw, each visible physel's screen rectangle is rasterised; each covered pixel casts its centre ray against the exact shape and writes the exact depth and the physel's index into a visibility buffer; shading runs once per pixel from the stored hit. Physels are culled in chunks of up to 64 in two rounds against a depth pyramid, and a far chunk is replaced by up to 8 fitted boxes only when its physels are under one pixel across and the boxes stray under half a pixel.

4. Detailed description

4.1 Words and frames

Word	Meaning
Physel	One record of solid matter: shape, place, materials
Object	Physels that share one frame (a position and a rotation). Objects move; their physels rest in them
Local	Rotated into the physel's axes and centred on it; unscaled, so distances are true

Word	Meaning
frame	
Unit frame	The local frame divided by the half-sizes, axis by axis. The core shape lies in the cube $[-1, 1]^3$. A ray's parameter s is the same in both frames
Profile axis	y. Taper narrows toward +y; the "ends" are the ±y faces, the "sides" the x–z cross-section

4.2 The record

The record is eight little-endian 32-bit words (Figure 1), read in one load. In every encoding below, round() takes the nearest integer and rounds halves away from zero (so 511.5 becomes 512).

Bytes	Field	Encoding
0-5	Position x, y, z	3 × u16: code = clamp(round(p/s) + 32768, 0, 65535); p = (code − 32768)·s, relative to the object's origin (the mean of its centres when made); s = E/32,000, E the largest coordinate distance of a centre from it (at least 1 mm): 0.03 mm steps when centres lie up to 1 m from the origin
6-9	Rotation	"Smallest three": i (2 bits) indexes the largest-magnitude component of the unit quaternion (x, y, z, w), made positive; the other three, in x, y, z, w order and each in $[-1/\sqrt{2}, 1/\sqrt{2}]$, take 10 bits: c = round((v·√2 + 1)/2·1023) clamped to 0-1023, v = (c/1023·2 − 1)/√2; the largest is $\sqrt{\max(0, 1 - v_0^2 - v_1^2 - v_2^2)}$. Packed u = i<<30 | c0<<20 | c1<<10 | c2
10-12	Half-sizes hx, hy, hz	3 × u8, logarithmic: h = exp(L_0 + b/255·(L_1 − L_0)), L_0 = ln 0.0005, L_1 = ln 8: half-sizes 0.5 mm to 8 m (physels 1 mm to 16 m across) in steps of a factor 1.0387 (3.9%); b = round((ln h − L_0)/(L_1 − L_0)·255), clamped to 0-255
13	Roundness	High nibble: sides exponent index (cross-section). Low nibble: ends exponent index (profile along y)
14	Taper T	0-255
15	Edge E	The bevel radius (§4.5)
16-17, 18-19	Materials A, B	u16 ids into the material table. A = 0xFFFF marks a carve (§4.9)
20	Blend	The share of B mixed into A, byte/255
21-22	Tint	Bit 15 picks the mode. 0, an offset from the material's colour: brightness in bits 0-5 (×0.6 to ×1.4), hue in bits 6-9 (±0.08 turn), saturation in bits 10-14 (×0.5 to ×1.5), each a linear code over its range; neutral is 0x4220. 1, an absolute colour: RGB 5:5:5 in bits 10-14, 5-9, 0-4
23	Seed	Per-physel variation (§4.4)
24	Flags	Bits 0-1 life (asleep, awake, dying); 2-3 current form (solid, liquid, granular, gas); 4 exposed (visible from outside its object); 5 burning; 6 in an object; 7 has an override
25, 26,	Temperature,	round((°C + 60)/10): −60 °C to 2,490 °C in 10 °C steps; 0-255; 0-255

Bytes	Field	Encoding
27	health, wetness	
28-31	Word 7	Zero (reserved, §4.3)

The words, with u the packed rotation:

```
w0 = px | py<<16            w1 = pz | (u & 0xFFFF)<<16       w2 = u>>16 | hx<<16 |
hy<<24
w3 = hz | roundness<<8 | taper<<16 | edge<<24                w4 = A | B<<16
w5 = blend | tint<<8 | seed<<24                               w6 = flags |
temperature<<8 | health<<16 | wetness<<24
w7 = 0
```

Full-precision state lives outside the record; bytes 25-27 mirror it. On a graphics card with 32-byte cache sectors a random 32-byte read touches one sector; 48 or 64 bytes would touch two.

The graphics card decodes a record into a 64-byte **instance** in world space, only when the record changes or its object moves, and keeps instances between frames:

```
decode(record, object position P, rotation Q, position step s):
    centre = P + rotate(Q, (codes of bytes 0-5 - 32768) * s)
    rot    = normalize(Q * unpack_smallest_three(bytes 6-9))
    h      = (size_dec(byte 10), size_dec(byte 11), size_dec(byte 12))
    instance = { centre, shape word = sides | ends<<4 | taper<<8 | bevel<<16;  rot;
                 h, material A | tint<<16;
                 object, seed | flags<<8 | temperature<<16 | wetness<<24, material B
| blend<<16, carve count }
```

4.3 An extended record

The extended record keeps 32 bytes and the bit positions of bytes 0-24 and 28-31 (Figure 1).

Bytes	Extended record
0-5	Position in the physel's innermost frame: a world tile, an object, a bone or a branch. Frames move; physels rest
16-19	Session ids from a dictionary of named materials in each world package, so packages from different makers cannot clash; 0xFFFF still marks a carve
24	Bits 0-1 life, 2-3 current form, 4 exposed; 5-7 free (burning moves to a record per face)
25-27	A 24-bit **state handle**. 0: no state of its own; read the coarser level (material, object, shared thermal node, place). Otherwise the bundle is at base + 32 × handle in one pool of 32-byte units: 2^{24} units address 512 MB; a 64-byte bundle takes two aligned units, a 128-byte one four. Run-time only: 0 in every file and hash
28-31	Word 7 (below). A word 7 of 0 decodes exactly as the record of §4.2

Word 7 bits	Field	Encoding
0-11	Size fine bits	4 bits per axis, signed −8..7: h = exp(L0 + (b + f/16)/255·(L1 − L0)) · 2^−k
12-17	Rotation fine	2 bits per stored component, signed −2..1: v = ((c + f/4)/1023·2 − 1)/√2

Word 7 bits	Field	Encoding
18-19	Shape family	0 dials only; 1 one unit-space cut; 2 a curved +y face; 3 a kind extension
20-31	Family parameters	1: a direction d (5 bits: one of 20 unit-space diagonals, (±1, ±1, 0) or (±1, ±1, ±1)) and an offset (7 bits); the solid keeps u·d ≤ o = (code − 64)·r/64, r = 2 or 3 (d's reach in the unit cube). 2: kx, kz (6-bit signed, in [−0.5, 0.5]); the top is y ≤ 1 − c(kx, x) − c(kz, z), c(k, u) = k·u² for k ≥ 0 (dome), |k|(1 − u²) for k < 0 (dish). 3: a kind id (bits 20-23) and kind byte (24-31); id 15 names a kind in a per-object block

Fine bits are found jointly, rounding first and splitting after (rotation: q = round(4x), coarse = floor((q + 2)/4), fine = q − 4·coarse; size: q = round(16x), coarse = floor((q + 8)/16), fine = q − 16·coarse). A reader that ignores word 7 is off by at most 0.625 of a rotation step and 0.53 of a size step. Word 7 cuts the size step from ±1.9% to ±0.12% (±0.6 mm, not ±9.5 mm, on a 1 m face) and the worst rotation error from about 0.24-0.27° to 0.06-0.07° (computed).

The size base per object. Each object stores k in 4 bits (0-15). Every half-size and every length the record stores is multiplied by 2^−k exactly. k = 0 is the range above; the smallest physel is 1 mm ÷ 2^k across: 7.8 μm at k = 7, 31 nm at k = 15. It costs one byte an object and nothing a physel. It only shrinks, so a reader that ignores k sees a box 2^k too big, never too small.

The superset rule. Whatever the family or kind, the three half-size bytes give a box that holds the shape, so any reader that ignores word 7 or k (culling, grids, chunk bounds, heat reach) stays safe.

4.4 Materials: stored once, two per physel

What a material is lives once, in a table indexed by material id (for example 256 entries): its look (16 bytes on the graphics card: linear base colour, roughness, metalness, refractive index, emissivity, detail, see-through flag and others), its rigid properties (density, strength, conductivity, melting, friction and others), its sound (Young's modulus, Rayleigh damping, absorption) and its behaviour as particles.

A physel's numeric properties mix linearly, X = (1 − β)·X_A + β·X_B with β = blend/255; yes-or-no properties come from the material with more than half. Shading mixes the looks the same way, applies the tint, and scales brightness by 1 + (seed/255 − 0.5)·2·detail; wetness darkens, and temperature adds thermal glow. Mortar into brick, or rust into steel, needs no extra physel.

4.5 The shape: one equation with four dials

Work in the unit frame (Figure 2).

1. **Exponents.** A nibble v (0-15) gives an exponent n. 0-7: n = 1 + v/7, from a diamond (n = 1) to round (n = 2). 8-14: 2.38, 2.88, 3.58, 4.63, 6.37, 9.84, 20.24, spaced so each step pulls a cube's corners in by the same distance (on a 1 m cube: 32.0, 27.5, 22.9, 18.3, 13.7, 9.2 and 4.6 cm). 15: n = ∞, a sharp edge, evaluated with max(), never as a large exponent. The sides nibble gives nS, the ends nibble nE.
2. **Taper.** t = 0.95·T/255. The width scale is w(y) = 1 − t(1 + y)/2: 1 at the bottom (y = −1), 1 − t ≥ 0.05 at the top. It is linear, so flat-sided and straight-sided shapes stay convex.
3. **Gauge.** x′ = x/w(y), z′ = z/w(y); g = (\|x′\|^nS + \|z′\|^nS)^(1/nS); N = (g^nE + \|y\|^nE)^(1/nE). The core K is the set N ≤ 1.
4. **Bevel.** r = (E/255)²·min(hx, hy, hz), in local units. The solid is K opened by a ball of radius r (Serra 1982): the union of all balls of radius r that fit inside K. Edges and corners sharper than r round to r; flat and gently curved parts do not move; parts thinner than 2r vanish; the bevel never makes a shape bigger. On a 1 m cube, E = 16 gives 2 mm, 64 gives 3 cm, 128 gives 13 cm and 255 gives 50 cm (a ball). The bevel applies where it has an exact distance: flat ends (nE = ∞) with square sides (nS = ∞), or with round sides (nS = 2) and hx = hz. Elsewhere E is ignored, and tools

write 0.

Each two-term norm is evaluated without overflow as m·((a/m)^p + (b/m)^p)^(1/p) with m = max(a, b), and as m at p = ∞. Powers apply to ratios at most 1, so exponents up to 20.24 are safe in 32-bit floats.

Convexity. Without taper N is a norm (both exponents are at least 1), so K is convex. With taper K stays convex when the ends are flat. With rounded ends a strong taper dents the side profile, exactly when t > 2(n − 1)/(n·2^(−1/n) + n − 1), n = nE: end nibble 0 dents at any taper (by up to 11.9% of the half-size at full taper); nibble 7 stays convex up to taper byte 222 (at most 2.3% beyond it); nibble 10 up to 250 (at most 0.09%); nibbles 11-15 never dent.

Kind	Dials	Ray test	Exact distance
Box	nS = nE = ∞, T = 0	Slab test	Box distance
Frustum (tapered box)	nS = nE = ∞, T > 0	Clip by 6 planes	Distance to a convex polytope
Ellipsoid	nS = nE = 2, T = 0	Quadratic	A lower bound
Cylinder (elliptic if hx ≠ hz)	nS = 2, nE = ∞, T = 0	Quadric side and 2 caps	Exact when hx = hz
Cone (truncated)	nS = 2, nE = ∞, T > 0	Quadric side and 2 caps	Exact when hx = hz
Rounded box, frustum, cylinder, cone	As above with E > 0	Newton steps on the exact distance	Distance to the shape eroded by r, minus r
General	Everything else	Guarded Newton steps on F (§4.6)	A lower bound (§4.7)

4.6 Ray casts

Setup. Move the ray into the local frame, relative to the physel's centre c (so precision does not depend on the distance from the world's origin), then into the unit frame, then clip it to the unit cube:

```
o_l = rotate(conj(q), o - c);   d_l = rotate(conj(q), d)
o_u = o_l / h;   d_u = d_l / h                    // per axis; the ray parameter s is unchanged
[b0, b1] = slab clip of o_u + s*d_u against [-1, 1]^3, intersected with [s_min, s_max]
if b1 <= b0: miss
```

Closed forms (unit frame). Box: the span is [b0, b1]. Ellipsoid: the roots of $\alpha s^2 + 2\beta s + \gamma = 0$ with $\alpha = |d_u|^2$, $\beta = o_u \cdot d_u$ and $\gamma = |o_u|^2 - 1$, in the stable form $q = -(\beta + \mathrm{sign}(\beta)\cdot\sqrt{(\beta^2 - \alpha\gamma)})$, roots q/α and γ/q. Cylinder and cone: the side is $x^2 + z^2 \le w(y)^2$, $w = a - b\cdot y$, $a = 1 - t/2$, $b = t/2$. Along the ray $w = c_0 + c_1 s$ with $c_0 = a - b\cdot o_y$ and $c_1 = -b\cdot d_y$; solve $q_a s^2 + 2q_b s + q_c = 0$ with $q_a = d_x^2 + d_z^2 - c_1^2$, $q_b = o_x d_x + o_z d_z - c_0 c_1$ and $q_c = o_x^2 + o_z^2 - c_0^2$; keep the interval where the quadric is negative and $w > 0$ (the mirror cone above $y = 2/t - 1$ lies outside the slab, but a steep ray can reach it); intersect with $|y| \le 1$. Frustum: clip by the six half-spaces $\pm x + b\cdot y \le a$, $\pm z + b\cdot y \le a$ and $\pm y \le 1$; the entry and exit planes give the normals.

Rounded kinds work in the local frame on the exact distance D of the bevelled shape. Box: with $v = |p| - (h - r)$, $D = |\max(v, 0)| + \min(\max(v_x, v_y, v_z), 0) - r$. Frustum: side planes $|x| \le c_x - k_x\cdot y$ with $k_x = h_x\cdot t/(2h_y)$ and $c_x = h_x(1 - t/2) - r\cdot\sqrt{(1 + k_x^2)}$ (likewise z), top at $\min(h_y - r, c_x/k_x, c_z/k_z)$, bottom at $-h_y + r$; inside, the largest plane distance; outside, the distance to the nearest face;

then minus r. Round cylinder or cone (h_x = h_z = R): the exact distance of the isosceles trapezoid in (ρ, y), ρ = √(x² + z²), with half-width R(1 − t/2) − r√(1 + k²) − k·y (k = R·t/(2h_y)) from y = −h_y + r up to h_y − r or its apex (Quilez), minus r. These shapes are convex, so D is convex along the ray, and a Newton step s ← s + D/(−D′) from outside never passes the surface (the walk below with q = 0, in world units).

General kinds: the solver function. In the unit frame let A = (|x|^nS + |z|^nS)^(1/nS) (the cross-section norm before the taper is divided out), B = w(y)·|y|, P = (A^nE + B^nE)^(1/nE), and

F = P − w(y) = w·(N − 1).

F is negative inside, positive outside, and zero on the same surface as N = 1, without N's infinitely steep parts near the tips. Along the ray

F′ = p_A·(∂A/∂x·d_x + ∂A/∂z·d_z) + (p_B·(sign(y)·w − |y|·t/2) + t/2)·d_y,

with ∂A/∂x = (|x|/A)^(nS−1)·sign(x) (likewise z), p_A = (A/P)^(nE−1) and p_B = (B/P)^(nE−1); at n = ∞ the derivative of max() picks the larger argument. The **guard** is q = ½·t·p_B·d_y².

Claim. For every δ, F(s + δ) ≥ F(s) + F′δ − qδ² (Figure 3). **Proof.** (1) A is a norm of (x, z), which are linear in s, so A(s + δ) ≥ A + A′δ. (2) B(y) = (1 − t/2)|y| − (t/2)·y·|y| has second derivative −t for y > 0 and +t for y < 0, and its kink at 0 bends upward, so B(s + δ) ≥ B + B′δ − (t/2)·d_y²·δ². (3) P is a norm of (A, B), hence convex, with a gradient (p_A, p_B) of non-negative parts, so P(A₁, B₁) ≥ P(A₀, B₀) + p_A(A₁ − A₀) + p_B(B₁ − B₀); insert (1) and (2). (4) w is linear in s: −w(s + δ) = −w(s) + (t/2)·d_y·δ. Adding gives the claim. ■

So the first crossing after s is no nearer than the smallest positive root of F + F′δ − qδ²:

Case	Step δ
q > 0	δ = 2F / (−F′ + √(F′² + 4qF)) (this form avoids cancellation)
q = 0, F′ < 0	δ = F/(−F′): plain Newton on a convex function
q = 0, F′ ≥ 0	No root: F only grows, so the ray misses

The walk (entry):

```
s = b0;  best = +infinity
repeat at most 32 times:
    (F, F', q, w) = solver values at o_u + s*d_u
    if F / w <= eps:                                  // N - 1 <= eps: a hit
        take at most 2 more guarded steps while F > 0  // polish; they never pass the
surface either
        return hit at s
    if F / w < best: best = F / w;  s_best = s
    delta = guarded step (table above);  if none: return miss
    s = s + delta;  if s > b1: return miss
return (best <= 4 eps) ? hit at s_best : miss         // at the cap only a grazing
ray counts
```

eps = δ_world/(√3·max(h_x, h_y, h_z)) turns the consumer's world tolerance (§4.8) into gauge units. The walk never samples and never takes a fixed step, so thin features (diamonds, sharp tapers, thin plates) cannot be stepped over.

Exits (for carves and see-through matter). Without a dent, the same walk runs backwards from b1. A dented shape is searched forward from the entry with a second bound from the same second derivative: on any stretch [a, b] of the ray, F ≤ max(F(a), F(b)) + (t/8)·(d_y(b − a))². Stretches whose bound stays within 2ε·w are proven inside and skipped; Newton probes look past the crossing; the first point more than ε outside brackets the exit, which is interpolated. At the cap of 64 steps the last point proven inside is returned: early, never past a dent.

The same procedure runs in 64-bit floats on the CPU (the reference) and in 32-bit floats on the graphics card, line by line.

4.7 Normals, volume, inside test and distance bounds

Normals are analytic: $\partial N/\partial x' = (g/N)^{(nE-1)}\cdot(|x'|/g)^{(nS-1)}\cdot\text{sign}(x')$ (likewise z') and $\partial N/\partial y = (|y|/N)^{(nE-1)}\cdot\text{sign}(y)$; at $n = \infty$, the face that max() selects. Through the taper, with $k = t/(2w^2)$:

$$\nabla N = (\partial N/\partial x' / w,\ \partial N/\partial y + x\cdot k\cdot\partial N/\partial x' + z\cdot k\cdot\partial N/\partial z',\ \partial N/\partial z' / w).$$

The local normal is normalise($\nabla N \oslash h$) ($\oslash$ divides per axis), rotated by q into the world. Bevelled kinds use the gradient of their exact distance; boxes the face's axis. Every cast also returns a **face code** (which plane, cap or branch of max() was hit, and which carve, if any), so the normal can be rebuilt later from the hit distance alone with the same choice.

Volume (closed form, bevel ignored):

$$V = h_x\cdot h_y\cdot h_z\cdot A_g\cdot[(1 - t/2)^2\cdot I_0 + (t/2)^2\cdot I_2],$$

with $A_g = 4\cdot\Gamma(1 + 1/nS)^2/\Gamma(1 + 2/nS)$ (the area of the unit cross-section), $I_0 = (2/nE)\cdot B(1/nE, 2/nE + 1)$ and $I_2 = (2/nE)\cdot B(3/nE, 2/nE + 1)$, B the Beta function; at $n = \infty$, $A_g = 4$, $I_0 = 2$ and $I_2 = 2/3$. **Derivation:** the cross-section at height y is the unit superellipse scaled by $w(y)\cdot(1 - |y|^{nE})^{(1/nE)}$; in $w^2 = (1 - t/2)^2 - t(1 - t/2)\cdot y + (t/2)^2\cdot y^2$ the odd term integrates to zero and the even moments are I_0 and I_2 (substitute $u = y^{nE}$). The **centroid** lies on the y axis at $y_c = -h_y\cdot t(1 - t/2)\cdot I_2 / [(1 - t/2)^2\cdot I_0 + (t/2)^2\cdot I_2]$: a tapered shape is heavier at its wide end. Mass is the blended density times V; the bevel's volume loss is ignored (under 1% for r below 5% of the size).

Inside test: $N \le 1$, or $D \le 0$ for bevelled kinds.

Distance. Exact for boxes and frustums, and for round-sided cylinders and cones with $h_x = h_z$, bevelled or not. Otherwise a lower bound: $D \ge (N - 1)\cdot\min(h)/(L\cdot J)$, where $L = 2^{\max(0, 1/nS - 1/2)}\cdot 2^{\max(0, 1/nE - 1/2)}$ bounds how fast the untapered gauge grows (a two-dimensional p-norm with $p < 2$ exceeds the Euclidean norm by at most $2^{(1/p - 1/2)}$), and $J = \sqrt{(2/w_{lo}^2 + 1 + t^2/(2w_{lo}^4))}$ bounds the taper's stretch where the width scale is at least w_{lo} (taken as half the width scale at the point).

4.8 Each consumer's error allowance

Consumer	Method	Allowed error
Main view	Closed form or walk, 32-bit floats	A quarter of a pixel at the hit: $\delta = \frac{1}{4}\cdot(2\cdot\tan(\theta/2)/H)\cdot\max(d_{entry}, 0.05\text{ m}) + 2^{-20}\cdot d_{centre}$ (θ the vertical field of view, H the image height in pixels, d_{entry} the distance to the physel's box entry, d_{centre} to its centre; the last term is a 32-bit float floor)
Shadow rays	The same shape, taper included, the same code	The shape's own: min(1 mm, 1% of the smallest half-size) + $2^{-20}\cdot\max(d_{entry}, 0.05\text{ m})$
Offline reference path tracer	The 64-bit code	10 μm or 10^{-5} of the largest half-size
Distance field (soft lighting, sound occlusion)	Exact distance where it exists, the lower bound elsewhere; carves as max(D_{solid}, $-D_{carve}$)	Never overstates distance; may understate by up to half a voxel. 5 cm voxels within ±20 cm of surfaces, rebuilt per 40 cm block where physics changed something

Consumer	Method	Allowed error
Far chunks	Up to 8 fitted boxes	Under half a pixel (§4.10)
Rigid contact	Support function of the convex hull: exact for untapered shapes through the dual norm (exponent n/(n − 1)), and for frustums and cones	1 mm or 1% of the smallest half-size

4.9 Objects and carves

An object is (∪ solids) − (∪ carves). A **carve** is an ordinary record whose material A is 0xFFFF, with any dials, taper and bevel; its other materials and state are ignored. A carve cuts only the solids of its own object (a cup's carve never cuts the table); carves do not cut each other and never add matter. Each solid lists at most 4 carves that overlap it. A cup is a cylinder, a carve and a handle.

Carve lists. One 16-byte header per 64 records holds, for each half of 32 records, a 32-bit mask of the carved records and the index of the first list. A carved record's list (four carve record numbers, padded with "none") is at first + popcount(mask & ((1 << bit) − 1)). The instance holds the carve count (0-4) in 3 bits.

```
cast_carved(solid, o, d):
    S = span(solid)                              // entry t0 and exit t1, with
normals
    C_k = span(carve k) for each listed carve    // [c0_k, c1_k]
    t = S.t0;  n = S.n0
    repeat at most 8 times:
        moved = false
        for each k: if c0_k <= t < c1_k: t = c1_k; n = -(exit normal of carve k);
moved = true
        if not moved: break
    if t >= S.t1: miss  else hit (t, n)
    exit, when wanted: the smaller of S.t1 and the first c0_k beyond t
```

Each primitive's first span is used; the second span of a dented solid is accepted as a small error. Inside test: inside some solid and inside no carve of that object. Distance: max(D_solid, −D_carve), which can understate but never overstates. A carved object's volume, centroid and mass are found once, when it is made or edited, by sampling the inside test. Carved objects get no stand-ins, since a box would fill their holes.

4.10 Drawing without meshes

Each frame, on the graphics card (Figure 4): decode changed records into instances; cull round 1 and draw it; build the depth pyramid; cull round 2 and draw it; stamp what is on screen; shade every pixel once.

Chunks (made on the CPU when an object is made, changes or breaks; Figure 5). Drawable physels (not carves; exposed flag set) are split recursively: sort the centres along the longest side of their bounding box and split at the multiple of 64 nearest the middle, until each group holds at most 64. Ties break by record number, so chunks never depend on listing order. A chunk holds its object, its members, a box in the object's frame (from each member's rotated reach |R·(h_x, 0, 0)| + |R·(0, h_y, 0)| + |R·(0, 0, h_z)|), a sphere about the box's centre holding every member's bounding sphere, the largest member radius |h|, and its stand-ins with their error. Chunks move with their object. The exposed flag is set when content is made, and on a physel's neighbours when breaking uncovers it; physels wholly inside their object are never drawn.

Stand-ins (made on the CPU with the chunks):

```
stand_ins(chunk):
    groups = the same split of its members into groups of ceil(n/8), each split along
```

```
the longest side
             in the frame of the member nearest the group's mean
    for each group:
        corners = the 8 box corners of every member
        for each distinct member rotation R: box_R = smallest box in R's axes holding
all corners
        box = the box_R of least volume
        box's thinnest half-size *= min(1, sum of member box volumes / box
volume)   // thin walls stay thin
        shares: each member adds (1 - blend) to material A and blend to material B
        A = largest share; B = second; blend = 255 * share_B / (share_A + share_B)
        tint, temperature, wetness, health = the group's means
    error = max over groups of: each member box corner's distance to the group's box,
and
            the distance of 5 x 5 points on each face of the box to the nearest
member box
    error = max(error, each member's slack)   // how far its box corner can lie from
its shape:
            (E/255)^2 * min(h) * 0.74 for a bevelled box; |h| - min(h) for other non-
box shapes
    if any member is see-through: error = infinity   // never used up close
```

The stand-in rule (graphics card, per chunk and frame). Let z be the chunk's nearest forward distance (its centre's forward distance minus its radius, at least the near plane), and $m = H/(2 \cdot z \cdot \tan(\theta/2))$ pixels per metre. Use the stand-ins if and only if $\text{error} \cdot m < p$ and $2 \cdot (\text{largest radius}) \cdot m < 2p$, with $p = 0.5$ pixel: every physel is under one pixel across and the stand-ins stray from them by under half a pixel. In a headset both eyes must agree.

Round 1 (one thread per chunk; then one workgroup per listed chunk, one thread per member):

```
for each chunk:
    skip if its sphere lies outside the four side planes of the view frustum
    apply the stand-in rule; if it holds, the chunk's members are its stand-ins
    if its sphere reaches the near plane: place all members now
    else if it was on screen last frame: place now only the members on screen last
frame;
                                          the others become physel candidates
    else: the chunk becomes a candidate
draw what was placed

place(physel, test):
    skip if removed or not exposed, or if its bounding sphere (radius |h|) is outside
the frustum
    if the camera is inside its box (grown by 0.2% and by twice the near distance):
list B (round 1 only)
    project the 8 corners of its box (grown by 0.1%):
        if any corner is nearer than 1.01 x the near plane: list A (round 1 only)
        lo, hi = min, max of the projected corners (normalised device coordinates)
        p0 = (lo + 1)/2 * size - 0.5;  p1 = (hi + 1)/2 * size - 0.5    // pixel
centres at whole numbers
        a = ceil(p0);  b = floor(p1), clamped to the screen
        if a > b on either axis: it covers no pixel centre: not drawn
        near = reversed depth of the nearest corner (near plane / forward distance)
        if test and hidden(a, b, near): skip
        append the rectangle (a, b, near, index): 16 bytes
```

The rectangle bounds the projection of the box, which holds the physel, so every pixel centre the physel covers lies inside it. A box's nearest point in depth is a corner, so near bounds the physel's depth.

The depth pyramid. Level l has one texel per 2^(l+1) pixels; each texel holds the farthest (smallest reversed) depth of the 2 × 2 below it, and texels past the screen's edge count as nothing drawn, so edges never hide anything. hidden(a, b, near) reads the depth buffer itself for rectangles up to 4 pixels a side, and otherwise the finest level whose texels are at least 1/k of the rectangle's size (at most $(k + 1)^2$ texels; k = 2 for physels, 6 for chunk boxes). It answers hidden when near is farther than every depth read.

Round 2: project each candidate chunk's box (turned with its object); skip it if it covers no pixel centre

or the pyramid hides it, else place its members with the test. Place each physel candidate with the test, and draw. Then **stamp**: each pixel that shows a physel marks that physel and its chunk with the frame number.

Nothing visible is skipped. A physel or chunk is skipped only if its box's nearest point lies behind what this frame has already drawn at every pixel the box could cover. Last frame's stamps choose only the order of drawing.

Drawing a rectangle: two triangles at the depth near, each vertex carrying the point one metre along its ray (which interpolates exactly). Lists A and B draw the box's 12 triangles, clipped by the hardware at the near plane. Each covered pixel centre runs:

```
d = normalize(ray through this pixel's centre)
hit = cast(physel, camera, d, the main view's tolerance)       // §4.6, with carves
(§4.9)
if not hit: discard
depth = near plane / forward distance of (camera + d * hit.t)  // reversed, 32-bit
float
visibility buffer = index + 1
hit buffer        = (hit.t as float bits, face code, flags, spare)
```

The exact hit is never nearer than the rectangle's depth, so the shader may declare its depth output conservative (only farther), and the ordinary depth test keeps the nearest exact hit in each pixel. The visibility buffer holds 32 bits a pixel: 0 for nothing, index + 1 for a physel or stand-in, other ranges for other kinds (particles, liquid surfaces). There is one exact cast at each pixel centre and no multisampling, so a physel that covers no pixel centre draws nothing; several casts per pixel, or exact coverage, may be used instead (§5).

Shading runs once per pixel: it rebuilds the point camera + t·d on the same ray and the normal from the shape at t with the stored face code (§4.7), without casting again, then applies the instance's materials (§4.4) and lights the point (see the related disclosures).

Overflow and headsets. A rectangle that finds no room in its buffer (sized from use) is recomputed by the vertex shader from the box's corners, with a margin of 1/64 pixel and its depth moved nearer by 10^{-5} of itself, so nothing the cast would keep is lost. A headset draws both eyes in one multiview pass; a physel goes on if either eye may see it, and each eye has its own rectangle, pyramid and stamps.

4.11 What runs where

Where and when	Steps
CPU, when content is made, changes or breaks	Encoding records; each object's origin and position step; chunks; stand-ins and their error; carve lists
Graphics card, each frame and view	Decoding changed records (compute); culling rounds, pyramid and stamps (compute); rectangles, boxes and exact casts (vertex and fragment stages); shading once per pixel (compute)
Graphics card, when physics changes a 40 cm block	The distance field

Everything uses standard Vulkan and SPIR-V features, with no maker-specific extensions.

5. Variants

The method may also use:

Record. Other sizes (16-64 bytes or more), variable lengths, split or structure-of-arrays layouts,

compressed groups. Position in 8-32 bits per axis, floats, or fixed point relative to a chunk, tile, bone or parent physel. Rotation in other bit counts ($2 + 3 \times 9$, $2 + 3 \times 15$), full quaternions, Euler angles, axis and angle, an index into common rotations, one per object, or none. Size linear, in floats, over other log ranges, with a base per object, chunk or tile, or fine bits elsewhere. One, three or more materials, several weights, a blend pattern (along an axis, from the seed) or a material per face. Other reserved values or flags for carves, media, visual-only matter and proxies. State in the record, a side array, a 16-32-bit handle, or a coarser level.

Shape. Other exponent tables (counts, spacings, continuous values, separate x and z exponents); curved, two-axis or any-axis taper; twist and bend (Barr 1984); linear, per-edge or chamfered bevels, and bevels on other kinds through distance bounds. More families through the same solver: supertoroids, round cones and capsules, primitives cut by planes, curved faces, height-grid tops, lattices and scatters inside one record, distance bricks, trilinear cells, polytopes, sweeps. **Flat (two-dimensional) physels**: a 2D region (rectangle, ellipse, superellipse, tapered, round-cornered or bent) of zero thickness for canvases, decals, text and menus, or of small thickness (for example 2 μm to 1 mm on a log scale) drawn as the exact thin slab, so a leaf still shows edge-on and drawing and physics share one shape. Other guarded steps: higher-order lower bounds; a guard from interval bounds on the second derivative; Lipschitz or segment bounds (Kalra and Barr 1989; Galin et al. 2020); bisection, regula falsi or interval arithmetic as fallbacks; other caps (8-128 steps); double, half or mixed precision; every span of a dented shape. Other tolerances: 1/16 to 1 pixel, the pixel's footprint, a world distance, a fraction of the physel's size, or one shared budget.

Objects and carves. 1-16 carves per solid, carves across objects, carves of carves, intersections, smooth unions, and rounding where a carve meets a solid.

Drawing. Tighter bounds (the projected hexagon or hull, ellipses for round kinds, Mara and McGuire 2013), point sprites, quads from mesh or geometry shaders, or one quad per chunk with a loop over its members. Casts in a compute shader (a software rasteriser binning rectangles into tiles) or on ray-tracing hardware, each physel or chunk a procedural bounding box whose intersection shader runs the same cast, for primary, shadow and reflection rays. A 64-bit visibility buffer kept by atomic minimum; forward shading; storing the hit distance, the face code, both or neither. Several exact casts per pixel averaged (supersampling), or exact coverage computed while drawing. Other culling (bounding volume hierarchies, octrees, portals, software occlusion, a reprojected pyramid), one round or several; other chunk sizes (8-1,024) and splits (surface area heuristic, Morton order, k-means). Other stand-ins (equal-volume ellipsoids or superquadrics, rounded boxes, images, points, coarser physels in a tree), thresholds (0.1-2 pixels), error measures and counts, and blended switches. Any graphics card, vector CPUs, integrated graphics, phones and standalone headsets, consoles, FPGAs or dedicated hardware; any graphics interface (Vulkan, Direct3D, Metal, OpenGL, WebGPU); one view, two eyes, many views (light-field and holographic displays) or offline frames.

Uses outside games:

Field	Use
CAD and engineering	Parts as unions and differences of exact primitives; exact mass properties; interference checks; huge assemblies without tessellation
Film and visual effects	Destruction in which every fragment stays an exact solid
Science and medicine	Tensor glyphs, molecules, cells, granular media; bones and organs from scans fitted to primitives
Robotics and collision	Robots and surroundings as primitives with exact distances and support functions; planning; training simulators
3D printing	Exact inside tests and volumes for slicing; lattices; material blends for multi-material printers

Field	Use
Digital twins, architecture, geology, mining, augmented reality	Buildings, rock and ground that can be dug, broken and measured; compact worlds over networks

6. Worked example

A truncated cone with a small bevel, upright at its object's origin:

Field	Chosen	Stored
Position	(0, 0, 0)	32768, 32768, 32768
Rotation	none (identity)	i = 3 and three codes of 512: u = 0xE0080200
Half-sizes	0.2, 0.5, 0.2 m	158, 182, 158 (decoding to 0.2013, 0.5007, 0.2013 m)
Roundness	round sides (7), flat ends (15)	0x7F
Taper	128	t = 0.4769: the top is 52.3% as wide as the bottom
Edge	32	r = (32/255)² × 0.2013 m = 3.2 mm (allowed: flat ends, round sides, h_x = h_z)
Materials	A = 0, B = 5, blend 64	25% of B
Tint, seed, flags	neutral, 200, exposed	0x4220, 200, 0x10
Temperature, health, wetness	20 °C, 255, 0	8, 255, 0

The 32 bytes: `00 80 00 80 00 80 00 02 08 E0 9E B6 9E 7F 80 20 00 00 05 00 40 20 42 C8 10 08 FF 00 00 00 00 00`.

The dials select a rounded cone, cast by Newton steps on its exact distance. By §4.7 ($A_g = \pi$, $I_0 = 2$, $I_2 = 2/3$) its volume is 0.07636 m³, the same as the frustum-of-a-cone formula $\pi H(R^2 + Rr + r^2)/3$ for these sizes, and its centroid lies 0.1012 m below its centre (computed, bevel ignored).

7. Figures



Figure 1. The record's byte map: the record of §4.2, the extended record of §4.3, and word 7's bits.

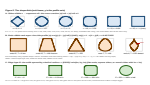


Figure 2. The shape dials: the sides exponent (A), the ends exponent and taper (B), and the bevel (C).

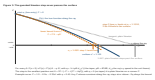


Figure 3. The guarded Newton step: F stays above the parabola $F + F'\delta - q\delta^2$, so its root is a safe step; a plain Newton step on the tangent would pass the surface.



Figure 4. From record to pixel: the steps each frame (left), and one physel's rectangle and per-pixel casts (right).



Figure 5. Chunks, stand-ins and the two culling rounds (left schematic: fewer physels a chunk than the 64 of §4.10).

8. References

- Aaltonen, S. (2018). GPU-based clay simulation and ray-tracing tech in Claybook. *Game Developers Conference 2018*.
- Barr, A. H. (1981). Superquadrics and angle-preserving transformations. *IEEE Computer Graphics and Applications*, 1(1), 11-23.
- Barr, A. H. (1984). Global and local deformations of solid primitives. *Computer Graphics (SIGGRAPH '84)*, 18(3), 21-30.
- Burns, C. A., and Hunt, W. A. (2013). The visibility buffer: a cache-friendly approach to deferred shading. *Journal of Computer Graphics Techniques*, 2(2), 55-69.
- Evans, A. (2015). Learning from failure (renderers for 'Dreams PS4'). *SIGGRAPH 2015 course: Advances in Real-Time Rendering in Games*.
- Galin, E., Guérin, E., Paris, A., et al. (2020). Segment tracing using local Lipschitz bounds. *Computer Graphics Forum*, 39(2), 545-554.
- Greene, N., Kass, M., and Miller, G. (1993). Hierarchical Z-buffer visibility. *Proceedings of SIGGRAPH '93*, 231-238.
- Gumhold, S. (2003). Splatting illuminated ellipsoids with depth correction. *Vision, Modeling, and Visualization 2003*.
- Haar, U., and Aaltonen, S. (2015). GPU-driven rendering pipelines. *SIGGRAPH 2015 course: Advances in Real-Time Rendering in Games*.
- Hart, J. C. (1996). Sphere tracing: a geometric method for the antialiased ray tracing of implicit surfaces. *The Visual Computer*, 12(10), 527-545.
- Hill, S., and Collin, D. (2011). Practical, dynamic visibility for games. In W. Engel (ed.), *GPU Pro 2*, A K Peters, 329-347.
- Hlawitschka, M., et al. (2008). Fast and memory efficient GPU-based rendering of tensor data. *IADIS International Conference on Computer Graphics and Visualization 2008*, 36-42.
- Jaklič, A., Leonardis, A., and Solina, F. (2000). *Segmentation and Recovery of Superquadrics*. Kluwer Academic Publishers.
- Kalra, D., and Barr, A. H. (1989). Guaranteed ray intersections with implicit surfaces. *Computer Graphics (SIGGRAPH '89)*, 23(3), 297-306.
- Karis, B., et al. (2021). A deep dive into Nanite virtualized geometry. *SIGGRAPH 2021 course: Advances in Real-Time Rendering in Games*.
- Kerbl, B., et al. (2023). 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4).
- Kindlmann, G. (2004). Superquadric tensor glyphs. *Proceedings of the Joint Eurographics–IEEE TCVG Symposium on Visualization*, 147-154.
- Knoll, A., et al. (2009). Fast ray tracing of arbitrary implicit surfaces with interval and affine arithmetic. *Computer Graphics Forum*, 28(1), 26-40.
- Laine, S., and Karras, T. (2010). Efficient sparse voxel octrees. *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*, 55-63.
- Luebke, D., et al. (2002). *Level of Detail for 3D Graphics*. Morgan Kaufmann.
- Maciel, P. W. C., and Shirley, P. (1995). Visual navigation of large environments using textured clusters. *Proceedings of the 1995 Symposium on Interactive 3D Graphics*, 95-102.
- Macklin, M., et al. (2014). Unified particle physics for real-time applications. *ACM Transactions on Graphics*, 33(4), 104.

- Mara, M., and McGuire, M. (2013). 2D polyhedral bounds of a clipped, perspective-projected 3D sphere. *Journal of Computer Graphics Techniques*, 2(2).
- Mitchell, D. P. (1990). Robust ray intersection with interval arithmetic. *Proceedings of Graphics Interface '90*, 68-74.
- Quilez, I. Distance functions. Web article, iquilezles.org.
- Roth, S. D. (1982). Ray casting for modeling solids. *Computer Graphics and Image Processing*, 18(2), 109-144.
- Serra, J. (1982). *Image Analysis and Mathematical Morphology*. Academic Press.
- Sigg, C., et al. (2006). GPU-based ray-casting of quadratic surfaces. *Eurographics Symposium on Point-Based Graphics*, 59-65.
- Toledo, R. de, and Lévy, B. (2004). Extending the graphic pipeline with new GPU-accelerated primitives. Technical report, INRIA.
- Zwicker, M., et al. (2001). Surface splatting. *Proceedings of SIGGRAPH 2001*, 371-378.

© 2026 Kyle Kimberlin. This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/